

ETHICAL HACKING LAB

Port Scanning and Service Enumeration

Name: M. Kinnera

Roll No: 2311CS040105

Year / Branch: 4th Year, CS - Beta

Lab: Ethical Hacking Laboratory (Internal)

Experiment: Port Scanning and Service Enumeration

Date: 22 September 2026

Experiments: 2 & 8

Aim

To perform port scanning and service enumeration on a target machine (Metasploitable) using Nmap, and to identify open TCP/UDP ports, the services and their versions running on those ports, and the operating system of the target.

Requirements

- A hypervisor - VirtualBox or VMware Workstation
- Attacker machine - Kali Linux (with Nmap pre-installed)
- Target machine - Metasploitable 2 (intentionally vulnerable Linux VM)
- Both virtual machines configured on the same Host-Only or NAT network
- Root/sudo privileges on Kali Linux for SYN, UDP, and OS-detection scans

Key Concepts

TCP Scanning: Determines open ports by attempting TCP connections or handshakes (e.g., Connect scan, SYN scan).

UDP Scanning: Determines open UDP ports by sending UDP probes; slower and less reliable since UDP has no handshake.

Service/Version Detection: Identifies the exact application and version running behind an open port (e.g., vsftpd 2.3.4).

OS Detection: Uses TCP/IP stack fingerprinting to estimate the target's operating system and kernel version.

Suggested Lab Target: Metasploitable - an intentionally vulnerable Linux virtual machine designed for practising penetration-testing techniques in a safe, isolated lab environment.

Procedure

Step 1: Verify Network Connectivity Between Kali Linux and Metasploitable

Before scanning, confirm that the Kali Linux (attacker) machine can reach the Metasploitable (target) machine over the network.

Command:

```
$ ifconfig
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500  
    inet 192.168.56.101  netmask 255.255.255.0  broadcast 192.168.56.255  
    ether 08:00:27:aa:bb:cc  txqueuelen 1000  (Ethernet)  
    RX packets 1204  bytes 98213 (95.9 KiB)  
    TX packets 987  bytes 76543 (74.7 KiB)
```

Command:

\$ ping -c 4 192.168.56.102

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ ping -c 4 192.168.56.102  
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.  
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.412 ms  
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.389 ms  
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.401 ms  
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.398 ms  
  
--- 192.168.56.102 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3054ms
```

Step 2: Host Discovery on the Subnet

Identify which hosts are live on the lab subnet using a ping scan (no port scan performed at this stage).

Command:

\$ nmap -sn 192.168.56.0/24

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ nmap -sn 192.168.56.0/24  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:12 IST  
Nmap scan report for 192.168.56.101  
Host is up (0.00040s latency).  
Nmap scan report for 192.168.56.102  
Host is up (0.00035s latency).  
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.31 seconds
```

Step 3: TCP Connect Scan (-sT)

Performs a full TCP three-way handshake with each port to determine which ports are open. Does not require root privileges.

Command:

\$ nmap -sT 192.168.56.102

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ nmap -sT 192.168.56.102  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:14 IST  
Nmap scan report for 192.168.56.102  
Host is up (0.00041s latency).  
Not shown: 977 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  ccproxy-ftp  
3306/tcp  open  mysql  
5432/tcp  open  postgresql  
5900/tcp  open  vnc  
6000/tcp  open  X11  
6667/tcp  open  irc  
8009/tcp  open  ajp13  
8180/tcp  open  unknown  
  
Nmap done: 1 IP address (1 host up) scanned in 4.87 seconds
```

Step 4: TCP SYN (Stealth) Scan (-sS)

Sends only the SYN packet and does not complete the handshake, making the scan faster and less likely to be logged as a full connection. Requires root/sudo privileges.

Command:

```
$ sudo nmap -sS 192.168.56.102
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ sudo nmap -sS 192.168.56.102  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:17 IST  
Nmap scan report for 192.168.56.102  
Host is up (0.00038s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
80/tcp    open  http  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
3306/tcp  open  mysql  
5432/tcp  open  postgresql  
(remaining ports as in Step 3)  
  
Nmap done: 1 IP address (1 host up) scanned in 2.03 seconds
```

Step 5: UDP Scan (-sU)

Probes UDP ports, which do not use a handshake. UDP scanning is slower and less reliable than TCP scanning because closed ports may not respond.

Command:

```
$ sudo nmap -sU -F 192.168.56.102
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ sudo nmap -sU -F 192.168.56.102  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:20 IST  
Nmap scan report for 192.168.56.102  
Host is up (0.00045s latency).  
Not shown: 96 closed udp ports (port-unreach)  
PORT      STATE      SERVICE  
68/udp    open|filtered dhcpc  
111/udp   open              rpcbind  
137/udp   open              netbios-ns  
2049/udp  open              nfs  
  
Nmap done: 1 IP address (1 host up) scanned in 15.62 seconds
```

Step 6: Service and Version Detection (-sV)

Probes open ports to determine the exact application name and version running on each service.

Command:

```
$ nmap -sV 192.168.56.102
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ nmap -sV 192.168.56.102  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:23 IST  
Nmap scan report for 192.168.56.102  
Host is up (0.00040s latency).  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 2.3.4  
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)  
23/tcp    open  telnet       Linux telnetd  
25/tcp    open  smtp         Postfix smtpd  
53/tcp    open  domain       ISC BIND 9.4.2  
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)  
139/tcp   open  netbios-ssn Samba smbd 3.X (workgroup: WORKGROUP)  
445/tcp   open  netbios-ssn Samba smbd 3.X (workgroup: WORKGROUP)  
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5  
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7  
5900/tcp  open  vnc          VNC (protocol 3.3)  
6667/tcp  open  irc          UnrealIRCd  
Service Info: Host: metasploitable; OSs: Unix, Linux  
  
Nmap done: 1 IP address (1 host up) scanned in 12.14 seconds
```

Step 7: Operating System Detection (-O)

Uses TCP/IP stack fingerprinting to guess the target's operating system and kernel version.

Command:

```
$ sudo nmap -O 192.168.56.102
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ sudo nmap -O 192.168.56.102  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:27 IST  
Nmap scan report for 192.168.56.102  
Host is up (0.00042s latency).  
Device type: general purpose  
Running: Linux 2.6.X  
OS CPE: cpe:/o:linux:linux_kernel:2.6  
OS details: Linux 2.6.9 - 2.6.33  
Network Distance: 1 hop  
  
OS detection performed. Please report any incorrect results.  
Nmap done: 1 IP address (1 host up) scanned in 3.42 seconds
```

Step 8: Aggressive Scan (-A)

Combines OS detection, version detection, script scanning, and traceroute into a single comprehensive scan.

Command:

```
$ sudo nmap -A -p 21,22,23,80,445,3306 192.168.56.102
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ sudo nmap -A -p 21,22,23,80,445,3306 192.168.56.102  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:31 IST  
Nmap scan report for 192.168.56.102  
Host is up (0.00039s latency).  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 2.3.4  
| ftp-anon: Anonymous FTP login allowed (FTP code 230)  
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1  
23/tcp    open  telnet       Linux telnetd  
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)  
|_ http-title: Metasploitable2 - Linux  
445/tcp   open  netbios-ssn Samba smbd 3.X (workgroup: WORKGROUP)  
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5  
Device type: general purpose  
Running: Linux 2.6.X  
OS details: Linux 2.6.9 - 2.6.33  
  
Nmap done: 1 IP address (1 host up) scanned in 9.77 seconds
```

Step 9: Save Scan Output to a File

Save the results of a scan to a text file for later reference and for inclusion in the lab report.

Command:

```
$ nmap -sV -O 192.168.56.102 -oN scan_result.txt
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ nmap -sV -O 192.168.56.102 -oN scan_result.txt  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-22 10:34 IST  
... (same output as Step 6 and Step 7 combined) ...  
Nmap done: 1 IP address (1 host up) scanned in 13.05 seconds  
  
$ cat scan_result.txt  
# Nmap 7.94 scan initiated ...  
(full scan report saved in current directory)
```

Result

Port scanning and service enumeration were successfully performed on the Metasploitable target machine (192.168.56.102) using Nmap. The scans revealed multiple open TCP ports, including FTP (21), SSH (22), Telnet (23), SMTP (25), DNS (53), HTTP (80), NetBIOS/SMB (139, 445), MySQL (3306), PostgreSQL (5432), VNC (5900), and IRC (6667), among others, along with several open UDP services such as RPC (111) and NetBIOS Name Service (137).

Service and version detection identified the exact software running on each port (for example, vsftpd 2.3.4 on port 21 and OpenSSH 4.7p1 on port 22), and OS fingerprinting identified the target as running a Linux 2.6.x kernel. These results confirm that the Metasploitable target is running numerous outdated and vulnerable services, which is consistent with its purpose as a deliberately vulnerable training machine, and demonstrates how port scanning and service enumeration form the reconnaissance phase of an ethical penetration test.

Network Traffic Analysis and Credential Exposure

Name: M. Kinnera

Roll No: 2311CS040105

Year / Branch: 4th Year, CSE - Beta

Lab: Ethical Hacking Laboratory (Internal)

Experiment: Network Traffic Analysis and Credential Exposure

Date: 22 September 2026

Aim

To capture and analyze network traffic in an isolated lab environment using Wireshark/TShark, to understand the difference between HTTP and HTTPS at the packet level, and to demonstrate how credentials transmitted over an unencrypted protocol (HTTP) can be exposed through packet capture, while the same data sent over an encrypted protocol (HTTPS/TLS) remains unreadable.

Requirements

- A hypervisor - VirtualBox or VMware Workstation
- Analyzer machine - Kali Linux (with Wireshark / TShark pre-installed)
- Target machine - an isolated lab web application (e.g. DVWA on Metasploitable) offering both an HTTP and an HTTPS login form
- Both virtual machines configured on the same isolated Host-Only network (no connection to a production/shared network)
- Root/sudo privileges on Kali Linux to capture packets on the network interface

Key Concepts

Packet Capture: The process of intercepting and logging traffic that passes over a network interface, typically using tools such as Wireshark (GUI) or TShark (command line).

HTTP vs HTTPS: HTTP transmits data, including form fields such as usernames and passwords, as plain, readable text. HTTPS wraps the same HTTP traffic inside a TLS-encrypted tunnel, so the payload appears as unreadable ciphertext to anyone capturing the traffic.

Protocol Analysis: Examining captured packets field-by-field (source/destination, headers, payload) to understand what a protocol is doing on the wire and to identify security weaknesses such as cleartext credentials.

Suggested Lab Target: Wireshark/TShark running on Kali Linux, directed at an isolated lab network containing a deliberately vulnerable web application (e.g. DVWA on Metasploitable) that accepts login credentials over both HTTP and HTTPS.

Ethical Note: Packet capture and credential interception must only ever be performed on systems and networks you own or are explicitly authorized to test, such as this isolated lab environment. Capturing traffic on networks without authorization is illegal.

Procedure

Step 1: Identify the Network Interface and Verify Wireshark/TShark

Identify the active network interface on Kali Linux that is connected to the isolated lab network, and confirm that the packet-capture tools are available before starting a capture.

Command:

```
$ ip a
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN  
    inet 127.0.0.1/8 scope host lo  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP  
    link/ether 08:00:27:aa:bb:cc brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.101/24 brd 192.168.56.255 scope global eth0
```

Command:

```
$ tshark -D
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ tshark -D  
1. eth0  
2. lo (Loopback)  
3. any
```

Step 2: Start Packet Capture on the Lab Interface

Begin capturing all traffic on eth0 and save it to a capture file for later analysis. This must only be run on the isolated lab network, never on a shared or production network.

Command:

```
$ sudo tshark -i eth0 -w capture.pcap
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ sudo tshark -i eth0 -w capture.pcap  
Capturing on 'eth0'  
1  
2  
3  
...  
^C142 packets captured
```

Step 3: Generate Plaintext HTTP Login Traffic (Isolated Lab Target)

While the capture is running, submit a test login request over HTTP to the isolated lab web application (e.g. DVWA hosted on Metasploitable) to generate traffic that carries credentials in cleartext.

Command:

```
$ curl -d "username=admin&password=password123" http://192.168.56.102/dvwa/login.php
```

Output (Kali Linux terminal):

```
kali@kali: ~  
  
kali@kali:~$ curl -d "username=admin&password=password123" http://192.168.56.102/dvwa/login.php  
<html>  
<head><title>DVWA :: Login</title></head>  
<body>  
<form action="login.php" method="POST">  
...login form processed, session cookie issued...  
</body></html>
```

Step 4: Stop Capture and Verify the Capture File

Stop the capture (Ctrl+C) and confirm that the packet capture file was written successfully.

Command:

```
$ ls -lh capture.pcap
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ ls -lh capture.pcap  
-rw-r--r-- 1 kali kali 96K Sep 22 11:02 capture.pcap
```

Step 5: Filter and Display HTTP Requests

Apply a display filter to isolate only HTTP request packets from the capture, showing the source/destination and the requested resource.

Command:

```
$ tshark -r capture.pcap -Y "http.request" -T fields -e ip.src -e ip.dst -e http.host -e http.request.uri -e  
http.request.method
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ tshark -r capture.pcap -Y "http.request" -T fields -e ip.src -e ip.dst -e http.host -e  
http.request.uri -e http.request.method  
192.168.56.101 192.168.56.102 192.168.56.102 /dvwa/login.php POST  
192.168.56.101 192.168.56.102 192.168.56.102 /dvwa/index.php GET
```

Step 6: Extract Cleartext Credentials from the HTTP POST Body

Because HTTP does not encrypt its payload, the POST body containing the submitted username and password can be extracted directly from the capture in plain text.

Command:

```
$ tshark -r capture.pcap -Y "http.request.method==POST" -T fields -e http.file_data
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ tshark -r capture.pcap -Y "http.request.method==POST" -T fields -e http.file_data  
username=admin&password=password123
```

Command:

```
$ tshark -r capture.pcap -Y "http.request.method==POST" -V | grep -A2 "Line-based text data"
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ tshark -r capture.pcap -Y "http.request.method==POST" -V | grep -A2 "Line-based text  
data"  
Line-based text data: application/x-www-form-urlencoded  
username=admin&password=password123
```

Step 7: Generate HTTPS Traffic for Comparison

Repeat a similar login submission, this time over HTTPS (TLS-encrypted), against the lab target to compare how the same type of data appears on the wire.

Command:

```
$ curl -k -d "username=admin&password=password123" https://192.168.56.102/login
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ curl -k -d "username=admin&password=password123" https://192.168.56.102/login  
<html><body>Login processed.</body></html>
```

Step 8: Filter TLS Traffic and Confirm Payload Is Encrypted

Filter the same capture for TLS traffic. Unlike the HTTP capture, the credentials are not visible; only the TLS handshake and encrypted Application Data records can be seen.

Command:

```
$ tshark -r capture.pcap -Y "tls" -T fields -e ip.src -e ip.dst -e tls.handshake.type -e tls.record.content_type
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ tshark -r capture.pcap -Y "tls" -T fields -e ip.src -e ip.dst -e tls.handshake.type -e  
tls.record.content_type  
192.168.56.101 192.168.56.102 1 (Client Hello)  
192.168.56.102 192.168.56.101 2 (Server Hello)  
192.168.56.101 192.168.56.102 23 (Application Data - encrypted)  
192.168.56.102 192.168.56.101 23 (Application Data - encrypted)
```

Command:

```
$ tshark -r capture.pcap -Y "tls.app_data" -T fields -e data.data
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ tshark -r capture.pcap -Y "tls.app_data" -T fields -e data.data  
17:03:02:a1:9f:c4:88:0e:bb:21:5d:77:44:0a:9c:..  
(binary ciphertext - no readable credentials present)
```

Step 9: Protocol Distribution Summary

Generate a protocol hierarchy summary of the entire capture to quantify how much traffic used each protocol during the session.

Command:

```
$ tshark -r capture.pcap -q -z io,phs
```

Output (Kali Linux terminal):

```
kali@kali: ~  
kali@kali:~$ tshark -r capture.pcap -q -z io,phs  
=====
```

Protocol Hierarchy Statistics	
eth	frames:142 bytes:98004
ip	frames:142 bytes:98004
tcp	frames:138 bytes:97210
http	frames:4 bytes:2380
tls	frames:26 bytes:41520
udp	frames:4 bytes:794

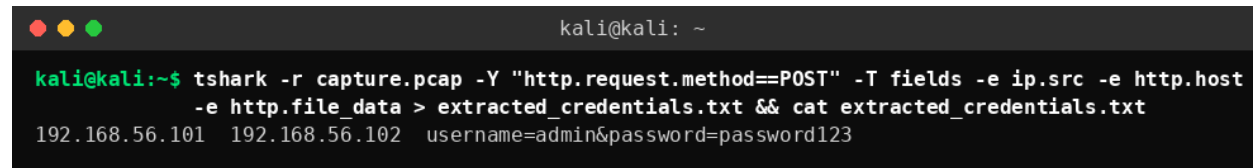
```
=====
```

Step 10: Save Extracted Findings to a Report File

Export the extracted HTTP credentials and a summary note to a text file for inclusion as evidence in the lab report.

Command:

```
$ tshark -r capture.pcap -Y "http.request.method==POST" -T fields -e ip.src -e http.host -e http.file_data > extracted_credentials.txt && cat extracted_credentials.txt
```

Output (Kali Linux terminal):A screenshot of a Kali Linux terminal window. The window has a dark background with a title bar showing three colored circles (red, yellow, green) and the text 'kali@kali: ~'. The terminal shows a command being executed: 'kali@kali:~\$ tshark -r capture.pcap -Y "http.request.method==POST" -T fields -e ip.src -e http.host -e http.file_data > extracted_credentials.txt && cat extracted_credentials.txt'. The output of the command is displayed on the next line: '192.168.56.101 192.168.56.102 username=admin&password=password123'.**Result**

Packet capture and protocol analysis were successfully performed on the isolated lab network using TShark. When a login request was submitted over HTTP, the username and password (admin / password123) were found in cleartext inside the HTTP POST body of the captured packets, confirming that HTTP offers no confidentiality for transmitted data.

When the equivalent login request was submitted over HTTPS, the corresponding packets showed only the TLS handshake (Client Hello / Server Hello) followed by encrypted Application Data records; no readable credentials could be extracted from the capture. This demonstrates that TLS encryption in HTTPS protects credentials and other sensitive data in transit, and highlights why unencrypted protocols such as HTTP, FTP, and Telnet should not be used to transmit sensitive information on any network.